

MTH 4300/4299: Algorithms, Computers, and Programming II

HW #5

Due Date: Dec 22nd, 2025

Problem 1

Modify the Binary Search Tree class to solve the following problems:

1. A member function to find the kth smallest element using an in-order traversal.
2. A member function to check if the tree is a valid BST using recursion.

A main function to:

1. Insert a set of values into the BST.
2. Prompt the user to input k and display the kth smallest element.(handle invalid values of k gracefully (e.g., display a message if k is out of bounds).)
3. Check if the tree is a valid BST and display the result.(A valid BST satisfies the property that for every node, all values in its left subtree are smaller, and all values in its right subtree are greater.)

Constraints:

1. The validation function should operate in $O(n)$ time complexity by performing a single traversal of the tree.
2. Assume $1 \leq k \leq n$, where n is the number of nodes in the BST.

Example:

Input:

Values to insert into BST: 20, 10, 30, 5, 15, 25, 35

Value of k: 3

Output:

In-order Traversal: 5 10 15 20 25 30 35

The 3rd smallest element is: 15

The tree is a valid BST.

Problem 2

To the binary search tree class from problem 1, add another method that prints out the post order of the nodes. Do not use recursion for this question and instead use a stack.

Problem 3

Write a C++ program that:

1. Reads data from a text file named **data.txt**. Each line in the file contains a student's name and their grades, separated by spaces.

Example file content:

```
Alice 85 90 78
Bob 70 88 82
Charlie 95 100 92
```

2. For each line:
 - Use a `stringstream` to parse the name and grades.
 - Compute the **average grade** for that student.
3. Write the results to a new file named `results.txt` in the following format:

```
Alice: 84.33
Bob: 80.00
Charlie: 95.67
```

Requirements

- Use `ifstream` to read from the input file.
- Use `ofstream` to write to the output file.
- Use `stringstream` to parse each line.
- Handle possible file opening errors.

Problem 4

You are given a working `BinaryTree` class that represents a general binary tree (not a binary search tree). The class already supports building the tree and basic traversals.

Your task:

Add the following method to the class:

```
bool isStructurallySymmetric();
```

This method should return `true` if the tree is **structurally symmetric**, and `false` otherwise.

Definition: A binary tree is *structurally symmetric* if the shape of the left subtree is a mirror image of the shape of the right subtree, **ignoring node values**.

Requirements

- You must use a **recursive helper function**.
- You may not create any additional data structures (no arrays, vectors, or queues).
- Do not modify any existing class methods.

Example

For the following tree:

```
    1
   / \
  2   3
   \ /
   4 5
```

The method call:

```
isStructurallySymmetric();
```

should return:

true

(The tree is symmetric in structure even though the values differ.)